

## QA Automation Knowledge Transfer (KT) – 2025

### 1. Technical Stack

- **Framework:** Playwright (Version 1.40+)
- **Language:** TypeScript
- **Runtime:** NodeJS 20.x
- **Test Runner:** Playwright Built-in Runner

#### Notes:

Playwright is chosen for its modern API, cross-browser support, and native parallel execution. TypeScript ensures type safety and maintainability.

---

### 2. Testing Scope

- **Primary Focus:** End-to-End (E2E) testing for the *Alpha* Web Portal.
  - **Integration Testing:** Covered via Playwright's API request context.
  - **Unit Testing:** Not included here. Refer to *Jest Framework Guidelines* for unit testing.
- 

### 3. Prohibited Tools & Practices

- **Selenium:** Strictly prohibited. All legacy Selenium scripts must be migrated to Playwright by Q3.
  - **Hardcoding:** No hardcoded credentials. Use `GlobalSetup.ts` for environment variables.
- 

### 4. Reporting

- **Tool:** Allure Reports
  - **Retention:** Reports stored in S3 bucket `qa-reports-prod` for 30 days.
  - **Access:** Reports are accessible via CI/CD pipeline artifacts.
- 

## 5. GenAI Integration

### 5.1 GitHub Copilot

- **Purpose:** Assists in writing Playwright scripts faster by suggesting code snippets.
- **Setup:**
  1. Install GitHub Copilot extension in VSCode.
  2. Authenticate with GitHub account.
  3. Enable Copilot for TypeScript projects.

- **Example Prompt:**
  - *“Write a Playwright test to log in to the Alpha portal and verify dashboard load.”*

## 5.2 Amazon Q Agent

- **Purpose:** Provides contextual answers and generates automation scripts using AWS Bedrock models.
  - **Setup:**
    1. Enable Amazon Q Agent in AWS Console.
    2. Connect repository (GitHub/CodeCommit).
    3. Configure IAM roles for secure access.
  - **Example Prompt:**
    - *“Generate Playwright script for validating API response in Alpha portal.”*
- 

## 6. Setup & Onboarding Steps

1. **Environment Setup**
    - Install NodeJS 20.x
    - Run `npm init playwright@latest`
    - Configure `tsconfig.json` for TypeScript
  2. **Project Structure**
  3. `/tests`
  4. `├─ e2e/`
  5. `├─ integration/`
  6. `/utils`
  7. `/config`
  8. `GlobalSetup.ts`
  9. **CI/CD Integration**
    - Use GitHub Actions with `playwright.yml` workflow.
    - Store reports in S3 via AWS CLI integration.
- 

## 7. Best Practices

- **Test Data Management:** Use JSON/YAML files for test data. Avoid inline hardcoding.
- **Code Reviews with AI:** Use Copilot or Amazon Q Agent to suggest improvements during PR reviews.

- **Security Considerations:**
    - Store secrets in AWS Secrets Manager.
    - Enforce MFA for CI/CD pipelines.
    - Validate API responses against schema.
- 

## **8. Conclusion**

This KT document provides a unified approach to QA automation using Playwright, enhanced by GenAI tools like GitHub Copilot and Amazon Q Agent. The combination of modern frameworks, AI-assisted development, and cloud-native reporting ensures scalability, maintainability, and faster onboarding for QA teams.